Qwen

From Performance to Score: A Transformer-Based Approach to Normalizing Solo Piano MIDI Performances

Foundational Methodologies for Expressivity Removal

The task of transforming an expressive musical performance into a structurally pure, metronomic score is a central challenge within Music Information Retrieval (MIR). This process involves systematically identifying and removing the nuanced elements of a performance—such as subtle timing variations, dynamic intensity changes, and rubato—to reconstruct the underlying notated music [42]. For the specific case of solo piano MIDI files, this problem requires sophisticated techniques capable of handling complex polyphonic textures where multiple notes overlap and interact. The foundational approaches to this problem can be broadly categorized into two domains: traditional, algorithmic methods inspired by digital audio workstation (DAW) functionalities, and more recent, model-based techniques that leverage machine learning. While DAW features provide an intuitive conceptual framework, they often lack the robustness required for automatic, high-fidelity normalization. In contrast, MIR research has developed more rigorous algorithms designed specifically for this purpose.

Traditional methodologies for rhythm normalization are conceptually analogous to the "quantization" feature found in mainstream DAWs like Logic Pro, Ableton Live, or Reason [34] [46] [48]. These tools operate on the principle of snapping note onsets to a predefined rhythmic grid. A user specifies a quantization value, such as "1/16th notes" or "Triplet 16th notes," and the software adjusts the timing of all selected notes to align with the nearest grid point. This directly addresses the issue of microtiming deviations, which are a hallmark of human expressive performance [7]. However, applying this technique automatically to a performance without a pre-defined grid requires first determining the underlying beat structure. This initial step, known as beat tracking, is non-trivial. It involves analyzing the performance's temporal structure to identify the recurring pulse, or beat, and its associated tempo [25]. Once the beat grid is established, a simple rule-based quantization algorithm can proceed by iterating through each note event in the

performance MIDI file. For each note's onset time, the algorithm calculates its position relative to the beat grid and moves it to the nearest grid-aligned timestamp. This process effectively erases the performer's fine-grained timing choices and replaces them with a rigid, metronomic timing structure. Similarly, duration normalization would involve calculating the difference between a note's new, quantized onset time and its original offset time, then rounding this duration to the nearest standard rhythmic value (e.g., quarter note, eighth note).

Despite their conceptual simplicity, these rule-based approaches have significant limitations when applied to complex polyphonic music. A primary challenge arises when multiple notes occur close together in time but belong to different metric levels or melodic lines. Simple snap-to-grid quantization lacks the context to resolve these ambiguities correctly. For example, if two notes start at times 100ms and 150ms, and the quantization grid is set to 1/16th notes (every 200ms), both notes would be snapped to the same starting time of 0ms, resulting in a loss of information about their relative order. This problem is exacerbated in dense textures common in solo piano repertoire. Furthermore, these methods struggle with performances exhibiting significant tempo fluctuations (rubato). A single, average tempo derived from the entire piece may not accurately represent the local tempos of different sections, leading to misalignment. While some advanced DAWs offer more sophisticated quantization parameters, such as swing factors or groove templates, these are typically designed to add, rather than remove, expressivity [12]. They do not inherently solve the core problem of extracting a single, unambiguous score from a variable-performance. The reliance on a fixed grid also means that the resulting score will only contain rhythmic values that correspond to that grid, potentially failing to capture more complex or compound meters accurately. Therefore, while foundational, these algorithmic methods often fall short of producing a musically coherent and structurally accurate normalized score from a complex performance.

Deep Learning Paradigms for End-to-End Conversion

Recent advancements in deep learning, particularly the development of transformer architectures, have introduced a paradigm shift in tackling the performance-to-score normalization task. Instead of relying on a series of discrete, hand-crafted algorithmic steps, these models frame the problem as a direct sequence-to-sequence translation task [14]. This approach offers a more holistic and powerful solution, capable of learning the intricate mappings between expressive performance data and structured symbolic

notation without requiring explicit alignment between a reference score and the performance. This methodology significantly reduces annotation costs associated with previous state-of-the-art pipelines, which heavily relied on complex Hidden Markov Model (HMM)-based systems for note-wise classification and alignment [14].

A seminal paper in this area proposes a novel deep learning approach that leverages a transformer architecture to convert expressive performance MIDI (P-MIDI) files directly into MusicXML scores [14]. The key innovation lies in its ability to bypass the need for a pre-existing canonical score, making it directly applicable to the user's goal of generating a normalized score from a raw performance file. The model processes the input P-MIDI and learns to predict the corresponding notated score, including not only pitches and durations but also higher-level notational details like trill marks, which was not a feature of prior works [14]. This end-to-end nature allows the model to implicitly learn the optimal rhythmic vocabulary and structural relationships inherent in the music, avoiding the rigidity of a pre-defined quantization grid. The success of this approach is rooted in several key architectural and methodological innovations described in detail in a subsequent paper [21].

One of the most critical components is the custom tokenization scheme, which serializes the symbolic music data into a format suitable for the transformer model. The system employs a parallel token stream paradigm, where each fundamental musical attribute—pitch, onset, duration, and velocity—is represented by its own stream of tokens [21]. Pitch is straightforwardly encoded using 128 distinct tokens corresponding to the MIDI note range. Onset and duration, being continuous time values, are converted into discrete tokens by binning their delta-time values into a fixed number of buckets; one study used 200 buckets after applying a logarithmic transformation (`-transform`) to better handle the wide range of possible inter-onset intervals [21]. Velocity is also quantized, mapped onto just 8 discrete tokens, which simplifies the prediction task while still capturing broad dynamic contours [21]. This "compound token" approach is highly efficient, reducing the overall sequence length by a factor of 3.5x compared to earlier serialization methods, making the training of large transformer models computationally feasible [14]. To ensure precise rhythmic reconstruction and avoid the drift issues common in purely delta-time encoding schemes, the model encodes absolute musical onsets using a hybrid approach. For each note, it predicts a `measure_offset` (its position relative to the start of its measure) and the `measure_length` (used for the first note in each measure) [21]. This allows the model to build the score's metrical structure from the ground up. Further enhancing rhythmic accuracy, the model's designers found that quantizing durations into 24th-note fractions (`durations_24`) yielded superior results compared to previous

methods that used powers-of-two denominators, achieving 98.6% coverage of the rhythmic values found in the ASAP dataset [21].

The effectiveness of this transformer-based approach is further bolstered by a comprehensive data augmentation strategy during training [21]. The model is exposed to a vast and varied dataset by applying transformations such as transposition (up to 12 semitones), global tempo scaling, adding random noise to note durations (duration jitter), and modifying the inter-onset intervals according to a specific formula (onset jitter). This forces the model to learn the invariant musical structures underlying the performance, rather than memorizing specific examples. During training, a greedy optimization strategy is employed to help the model align performed notes with beat-level annotations; if a note falls within 50ms of a beat, it may be moved to the preceding or next inter-beat interval if doing so minimizes mismatched pitches between the performance and the target score, improving alignment in complex situations like rapid passages or trills [21]. This sophisticated combination of a sequence-to-sequence architecture, efficient tokenization, robust timing encoding, and extensive data augmentation represents the current state-of-the-art for generating a normalized, metronomic score from an expressive MIDI performance, directly addressing the requirements of the research goal.

Feature	Traditional Algorithmic Approach	Deep Learning (Transformer) Approach
Core Principle	Snapping note onsets to a pre-defined rhythmic grid. [34]	End-to-end sequence-to-sequence translation from performance to score. [14]
Beat Detection	Requires external beat tracking to establish the grid. [25]	Implicitly learns beat and downbeat structure from data. [6] [22]
Rhythmic Vocabulary	Fixed by the chosen quantization grid (e.g., 16th notes).	Learned from data; can discover and use a diverse set of rhythmic values. [21]
Polyphony Handling	Ambiguous; can lead to incorrect chord formation.	Handles polyphony naturally through its sequence modeling capabilities. [21]
Score Requirement	Not required; operates on performance data alone.	Not required; frames the problem as a direct translation task. [14]
Complexity	Conceptually simple, algorithmically straightforward.	Highly complex architecture requiring significant computational resources and expertise.

Technical Implementation and Polyphonic Reconstruction

Implementing a system to normalize a solo piano MIDI performance into a metronomic score requires careful consideration of the available tools and the specific challenges posed by polyphony. The choice of library for MIDI manipulation is paramount, with several Python packages offering powerful functionalities for parsing, analyzing, and generating MIDI data. Libraries such as `miditoolkit`, `pretty_midi`, and `partitura` are frequently cited in the literature as essential tools for MIR tasks [54] [58] [69]. `Miditoolkit`, for instance, provides a user-friendly interface built upon the lower-level `mido` library, allowing developers to easily load MIDI files, iterate through events, and construct musical objects [54]. Similarly, `partitura` is an object-oriented library designed for symbolic music processing, supporting import and export from formats like MusicXML and MIDI, and providing robust tools for handling musical data [52] [58]. These libraries abstract away much of the low-level complexity of the MIDI file format, enabling researchers and developers to focus on the core logic of the normalization algorithm.

Regardless of the chosen methodology—whether a multi-stage algorithmic pipeline or a deep learning model—the ultimate goal is to produce a 'score' formatted MIDI file using the MIDI.py module. According to the provided documentation, the 'score' format is characterized by the use of absolute timing for all events [42]. This contrasts with other formats like 'opus', which use delta-times (the time elapsed since the previous event). Working with the 'score' format implies constructing a final list of events where each event contains an absolute timestamp indicating precisely when it should occur in the timeline. This format is ideal for representing a notated score, as it explicitly defines the simultaneous occurrence of multiple notes within a single time slice. The implementation must therefore process the input MIDI file to extract all note events (both `note_on` and `note_off`), apply the normalization transformations, and then synthesize a new list of events with corrected absolute timestamps before writing it out.

The most significant technical hurdle in this process is the accurate reconstruction of polyphonic textures. In a solo piano performance, it is common for chords to be played simultaneously, and for melodies to weave in and out of dense accompaniment patterns. An effective normalization algorithm must correctly identify which notes are sounding together at any given moment. In a deep learning approach, this is handled implicitly by the model's attention mechanisms, which learn to group notes that co-occur in time and space [21]. However, in a multi-stage algorithmic pipeline, this requires a deliberate and robust procedure. After quantizing the onset times of all notes, the system must iterate

through the timeline and, at each quantized time step, collect all active notes. A note is considered "active" if its newly quantized start time is less than or equal to the current time step and its calculated end time is greater than the current time step. These collected notes form a chord, which should be represented in the output score as a single `note_on` event containing multiple pitches at that specific timestamp. The corresponding `note_off` event for the entire chord would then be placed at the earliest future time step where all constituent notes end. This process must be repeated for every unique quantized time step in the performance.

The following table outlines a potential workflow for an algorithmic implementation:

Step	Action	Key Considerations
1. Input Parsing	Load the input MIDI file using a library like `miditoolkit` or `partitura`. [54] [58]	Extract all `note_on` and `note_off` events, pairing them by pitch and track to create `(pitch, start_time, end_time)` tuples for each note.
2. Beat Tracking	Analyze the performance to determine the underlying beat grid and tempo map. [25]	Use a dedicated beat-tracking algorithm. This establishes the fundamental timing units for quantization. The output is a sequence of beat times.
3. Onset Quantization	For each note's `start_time`, find the closest beat grid position and assign it as the new, quantized `start_time`. [34]	This is the primary mechanism for removing microtiming and creating a metronomic feel.
4. Duration Normalization	Calculate the note's duration (`end_time - start_time`). Round this value to the nearest standard rhythmic value. [21]	This determines the final notated duration of the note in the normalized score.
5. Polyphonic Reconstruction	At each unique quantized `start_time`, collect all notes whose new `start_time` is <= current time and whose `end_time` > current time. [21]	This forms the chords. Group these notes logically for the output score.
6. Output Generation	Construct a list of 'score' format events with the new absolute timings. Use MIDI.py functions to write this list to a new MIDI file. [42]	Ensure `note_on` events for all pitches in a chord occur at the same timestamp. Place `note_off` events at the correct future timestamps.

This systematic approach ensures that even complex polyphonic passages are deconstructed and reassembled into a clean, metronomic representation. The final output will be a MIDI file that captures the harmonic and melodic content of the original performance but stripped of its performative expression, adhering strictly to the rhythmic constraints defined by the beat grid.

Synthesis of Findings and Recommended Pathways

The research into normalizing expressive MIDI performances reveals a clear trajectory from traditional, rule-based algorithms towards sophisticated, end-to-end deep learning models. For the specific goal of generating a completely normalized metronomic score from a solo piano MIDI file, the latest and most effective approaches reside within the domain of transformer-based sequence-to-sequence architectures. These models, exemplified by the work presented in "Music Generation with Transformers" [21], offer a comprehensive solution that directly addresses all facets of the user's request: removing expressivity, ensuring metronomic precision, and correctly handling complex polyphony. By framing the problem as a direct translation from performance data to a symbolic score, they circumvent the need for cumbersome alignment procedures and fixed quantization grids, instead learning the underlying musical structure from data [14]. Their ability to implicitly learn beat and downbeat information, coupled with innovative tokenization schemes that encode absolute timing, makes them exceptionally well-suited for this task [21].

However, the implementation of such a deep learning model represents a significant undertaking. It requires expertise in machine learning frameworks like PyTorch or TensorFlow, substantial computational resources for training, and access to a large, curated dataset of matched performance and score pairs, such as those found in collections like PianoCoRe or GiantMIDI-Piano [5] [11]. The provided sources indicate that while the architectural blueprint is well-defined, a publicly available, pre-trained model implementing this exact approach is not referenced, meaning that achieving this highest fidelity result would likely involve either training a custom model or adapting existing open-source code. This path offers the greatest potential for accuracy and robustness but comes with a high barrier to entry.

For users seeking a more pragmatic and accessible pathway, a multi-stage algorithmic pipeline presents a viable alternative. This approach breaks the problem down into manageable steps: beat tracking, onset quantization, duration normalization, and polyphonic reconstruction. By leveraging established MIR libraries such as `miditoolkit` or `partitura` for MIDI manipulation and specialized tools for beat tracking, a developer can construct a transparent and interpretable system [25] [54] [58]. While this method may not match the nuance and contextual understanding of a trained neural network, it provides a high degree of control over the normalization process. The user can explicitly define the quantization grid (e.g., 16th notes, 32nd note triplets) and carefully manage the logic for grouping notes into chords at each time step. This method is more susceptible to errors in ambiguous polyphonic passages but remains a powerful

tool for achieving a reasonable level of metronomic normalization. The choice between these two pathways ultimately depends on the user's specific needs regarding accuracy, available resources, and tolerance for complexity. For the stated goal of a complete, clear, and concise implementation, a well-documented algorithmic pipeline may offer the most direct route to a functional solution.

Reference

1. SuperCollider: A Creative Music Guide | PDF - Scribd https://www.scribd.com/document/727071030/019761700X
2. [PDF] Transformer-Based Rhythm Quantization of Performance MIDI Using ... https://arxiv.org/pdf/2604.22290
3. MIR-1K https://opendatalab.com/OpenDataLab/MIR-1K
4. Transformer-Based Rhythm Quantization of Performance MIDI Using ... https://arxiv.org/html/2604.22290v1
5. PianoCoRe: Combined and Refined Piano MIDI Dataset - arXiv https://arxiv.org/html/2605.06627v1
6. (PDF) Beat-Based Rhythm Quantization of MIDI Performances https://www.researchgate.net/publication/395034307_Beat-Based_Rhythm_Quantization_of_MIDI_Performances
7. Expressiveness in metronomic rolls for player piano: A new field of ... https://www.sciencedirect.com/science/article/pii/S1296207424000852
8. Score Images as a Modality: Enhancing Symbolic Music ... - MDPI https://www.mdpi.com/1424-8220/24/15/5017
9. Multi-modal music techniques for synthesizing high-quality audio ... https://www.nature.com/articles/s41598-025-17410-6
10. Score-Informed Transformer for Refining MIDI Velocity in Automatic ... https://arxiv.org/html/2508.07757v2
11. (PDF) GiantMIDI-Piano: A Large-Scale MIDI Dataset for Classical ... https://www.researchgate.net/publication/360558562_GiantMIDI-Piano_A_Large-Scale_MIDI_Dataset_for_Classical_Piano_Music
12. Using Quantec Room Simulator for Surround Sound in Logic Pro for ... https://support.apple.com/en-is/guide/logicpro/lgcpba0394ac/12.2/mac/15.6
13. Foundation Models for Music: A Survey - arXiv https://arxiv.org/html/2408.14340v1

14. [2410.00210] End-to-end Piano Performance-MIDI to Score ... - arXiv https://arxiv.org/abs/2410.00210
15. Estimating Onset and Offset Asynchronies in Polyphonic Score ... https://www.researchgate.net/publication/265647225_Estimating_Onset_and_Offset_Asynchronies_in_Polyphonic_Score-Audio_Alignment
16. [PDF] pyAMPACT: A Score-Audio Alignment Toolkit for Performance Data ... https://arxiv.org/pdf/2412.05436
17. Estimating Onset and Offset Asynchronies in Polyphonic Score ... https://www.semanticscholar.org/paper/Estimating-Onset-and-Offset-Asynchronies-in-Devaney/f58eeb0a50e977639b22df3f146492ab5ab36c21
18. [PDF] Results from the 2025 AMT Challenge - OpenReview https://openreview.net/pdf?id=NG187AZ71W
19. Advancing Multi-Instrument Music Transcription: Results from ... - arXiv https://arxiv.org/html/2603.27528v1
20. [PDF] arXiv:2504.15071v1 [cs.SD] 21 Apr 2025 https://arxiv.org/pdf/2504.15071
21. End-to-End Piano Performance-MIDI to Score Conversion ... - arXiv https://arxiv.org/html/2410.00210v1
22. Beat-Based Rhythm Quantization of MIDI Performances - arXiv https://arxiv.org/html/2508.19262v1
23. [PDF] Beat-Based Rhythm Quantization of MIDI Performances - arXiv https://arxiv.org/pdf/2508.19262
24. [2508.19262] Beat-Based Rhythm Quantization of MIDI Performances https://arxiv.org/abs/2508.19262
25. Beat and Downbeat Tracking in Performance MIDI Using an End-to ... https://arxiv.org/html/2507.00466v1
26. Research in Computational Expressive Music Performance and ... https://www.mdpi.com/2414-4088/7/2/15
27. [PDF] arXiv:2408.04309v1 [cs.SD] 8 Aug 2024 https://arxiv.org/pdf/2408.04309
28. Human newborns form musical predictions based on rhythmic but ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12875487/
29. [PDF] Cortical and behavioral tracking of rhythm in music: Effects of pitch ... https://nyaspubs.onlinelibrary.wiley.com/doi/am-pdf/10.1111/nyas.15315
30. Communicating Sound Through Natural Language - arXiv https://arxiv.org/html/2605.08750v1
31. DRUMS: Drummer Reconstruction Using Midi Sequences https://dl.acm.org/doi/10.1145/3769047.3769066

32. Foundry Local 1.1: Live Transcription, Embeddings, and Responses ... https://devblogs.microsoft.com/foundry/foundry-local-v1-1/

33. [PDF] NLP4MusA 2026 The 4th Workshop on NLP for Music and Audio ... https://aclanthology.org/2026.nlp4musa-1.pdf

34. Logic Pro advanced quantization parameters - Apple-kundestøtte (NO) https://support.apple.com/no-no/guide/logicpro/lgcp35029a2b/10.7/mac/11.0

35. Route MIDI internally to software instrument tracks in Logic Pro for Mac https://support.apple.com/guide/logicpro/route-midi-internally-software-instrument-lgcp1efa7c4d/mac

36. Logic Pro beat mapping overview - Apple সহায়তা (IN) https://support.apple.com/bn-in/guide/logicpro/lgcp624214db/10.7/mac/11.0

37. [PDF] arXiv:2304.12939v1 [cs.SD] 24 Apr 2023 https://arxiv.org/pdf/2304.12939

38. Capstone-Assignment - RPubs https://rpubs.com/BushraT/1075426

39. Musicaiz: A python library for symbolic music generation, analysis ... https://www.sciencedirect.com/science/article/pii/S2352711023000614

40. [PDF] Fine-Grained Position Helps Memorizing More, a Novel Music ... https://ojs.aaai.org/index.php/AAAI/article/view/25650/25422

41. End-to-End Full-Page Optical Music Recognition for Pianoform ... https://link.springer.com/article/10.1007/s11263-025-02654-6

42. Decoding the evolution of melodic and harmonic structure of ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC13106658/

43. Note-Level Singing Melody Transcription for Time-Aligned Musical ... https://arxiv.org/html/2502.12438v1

44. Machine Learning Techniques in Automatic Music Transcription - arXiv https://arxiv.org/html/2406.15249v1

45. Cortical and behavioral tracking of rhythm in music - PMC - NIH https://pmc.ncbi.nlm.nih.gov/articles/PMC11998481/

46. Live 12 Beta Release Notes - Ableton https://www.ableton.com/en/release-notes/live-12-beta/

47. [PDF] Proceedings of the 19th Linux Audio Conference - HAL https://hal.science/hal-05194352v1/file/proceedings.pdf

48. Reason 12 User Manual Overview | PDF | Synthesizer - Scribd https://www.scribd.com/document/779280791/Reason-12-Operation-Manual

49. Using Quantec Room Simulator for Surround Sound in Logic Pro for ... https://support.apple.com/en-ng/guide/logicpro/lgcpba0394ac/mac

50. Mixbus 11 User Manual Overview | PDF | Sound Recording - Scribd https://www.scribd.com/document/898344459/MixBUS-v11-Manual
51. [PDF] Release 1.0.0 The mirdata development team https://mirdata.readthedocs.io/_/downloads/en/1.0.0/pdf/
52. An Introduction to Symbolic Music Processing with Partitura https://partitura.readthedocs.io/en/latest/Tutorial/notebook.html
53. [PDF] A PYTHON PACKAGE FOR SYMBOLIC MUSIC PROCESSING - arXiv https://arxiv.org/pdf/2206.01071
54. miditoolkit - PyPI https://pypi.org/project/miditoolkit/
55. Robust Polyphonic Midi Score Following with Hidden Markov Models https://www.researchgate.net/publication/228716894_Robust_Polyphonic_Midi_Score_Following_with_Hidden_Markov_Models
56. [PDF] musif: a Python package for symbolic music feature extraction - arXiv https://arxiv.org/pdf/2307.01120
57. How to split midi file based on notes pitch using python? https://stackoverflow.com/questions/63979920/how-to-split-midi-file-based-on-notes-pitch-using-python
58. [PDF] partitura: A Python Package for Handling Symbolic Musical Data https://arxiv.org/pdf/2201.13144
59. [PDF] arXiv:2202.06034v2 [cs.SD] 21 Feb 2022 https://arxiv.org/pdf/2202.06034
60. [PDF] Compilation of real-time audio DSP on FPGA - HAL theses https://theses.hal.science/tel-05033875/file/these.pdf
61. Follow rhythm of chords and other tracks in Logic Pro for Mac https://support.apple.com/guide/logicpro/follow-rhythm-of-chords-and-other-tracks-lgcpca8dca3d/mac
62. english-words.txt - Miller https://miller.readthedocs.io/en/6.13.0/data/english-words.txt
63. [2011.06801] A Comprehensive Survey on Deep Music Generation https://ar5iv.labs.arxiv.org/html/2011.06801
64. Utils methods — MidiTok 2.0.7 documentation https://miditok.readthedocs.io/en/v2.0.7/utils.html
65. A Survey of Music Generation in the Context of Interaction - arXiv https://arxiv.org/html/2402.15294v1
66. [PDF] Note-Level Singing Melody Transcription for Time-Aligned Musical ... https://arxiv.org/pdf/2502.12438
67. Polyphonic Pitch Tracking with Deep Layered Learning https://www.researchgate.net/publication/324387815_Polyphonic_Pitch_Tracking_with_Deep_Layered_Learning

68. [PDF] Zero-Note Samba: Self-Supervised Beat Tracking - HAL https://hal.science/hal-03669865v2/file/desblancs2022taslp_zero-note-samba.pdf
69. A Survey on Deep Learning for Symbolic Music Generation https://dl.acm.org/doi/10.1145/3597493
70. (PDF) Polyphonic Music Generation with RNN - ResearchGate https://www.researchgate.net/publication/356602705_Polyphonic_Music_Generation_with_RNN
71. A Survey on Deep Learning for Symbolic Music Generation https://dl.acm.org/doi/pdf/10.1145/3597493
72. Midi - how to get all notes for specific second(s) - Stack Overflow https://stackoverflow.com/questions/60033297/midi-how-to-get-all-notes-for-specific-seconds
73. (PDF) Audio-To-Score Alignment Of Piano Music Using RNN-Based ... https://www.researchgate.net/publication/317840831_Audio-To-Score_Alignment_Of_Piano_Music_Using_RNN-Based_Automatic_Music_Transcription